PARADIS with spherical DISCO convolutions

Native PyTorch port and compact CPU reproduction

Reproduction workspace: `yikwill/paradis-spherical`

October 2026

Abstract

This work ports the public PARADIS implementation to native PyTorch, replaces all spatial convolutions with selectable DISCO operators, removes learned spatial bias maps, and restricts explicit coordinate inputs to unnormalized sine of latitude. The unchanged upstream trainer and the native orchestration agree exactly on a small deterministic output/loss/gradient/AdamW fixture. Additional checks cover spherical operators, temporal splits, autoregression, checkpoint continuation, distributed CPU execution, and real-data forecast writing. Experiments use a full 721×1440 tracer grid and a compact, paired 2,000-update weather comparison on real ERA5. Fixed-target tracer fits fail; identity warmup and gradual displacement give PARADIS relative MSE 0.00232 with signed mass error +8.61%. The final DISCO/Conv2d weather-loss ratio is 0.9995, and the joint CPU no-slowdown criterion passes. These bounded experiments do not reproduce the paper's full weather training or establish GPU performance. Failed targets and all attempted tracer protocols are retained.

1 Scope and source

The reference is Pereira et al.'s *Learning to Advect*, version 1, whose Figure 3 compares a depth-three U-Net to a 20-parameter neural semi-Lagrangian (NSL) operator [1]. As requested for this fork, the replacement panel uses the complete compact PARADIS architecture, including its encoders, two advection–diffusion–reaction blocks, gates, and decoder. It is therefore a modified experiment, not an exact reconstruction of the paper's small NSL model.

The full upstream history is retained at revision `9a6f4b7d0dadfb80918ff8929bd92b6a5da2bcc5`. The unchanged reference is a detached worktree. The primary implementation commit is `58f5fbf`; the folded CPU kernel and its checks are committed as `9a235ec`. Other commits contain additional checks, experiment drivers, measurements, and this report. Saved weather configurations and provenance files identify the training source. The private repository is https://github.com/yikwill/paradis-spherical.

The accepted budget is 12 CPU experiment wall-hours, excluding installation, data download, and report compilation. Four GPU-hours remain reserved because supported GPU hardware was unavailable. The execution ledger in `results/budget.json` records a conservative wall-time upper bound and individual run durations. Its interval includes setup and reporting to avoid undercounting and is below four hours. Concurrent run durations must not be summed as elapsed wall time.

2 Implementation and numerical verification

2.1 Native orchestration

The native trainer retains the upstream autoregressive feature layout, loss scale, truncated backpropagation, fixed accumulation denominator, AdamW parameter groups (including the gate learning-rate multiplier), and warmup–steady–decay schedule. It implements validation, clipping, early stopping, atomic checkpoints, and `torchrun` distribution without Lightning or TensorBoard. Checkpoints save optimizer/scheduler state, random generators, progress, grid, feature order, and normalization metadata. Exact continuation rejects incompatible training settings or distributed world size. Forecast horizons can change without changing weights.

Static encodings are reused within a truncation chunk and across a forecast through an explicit argument. This avoids caching activations across unrelated batches. The data cache holds raw arrays; every sample is copied before normalization. Preloaded sample access bypasses xarray/Dask indexing and reuses time/solar forcings. A configured memory budget bounds preload allocation.

Check	Verified result
Unchanged upstream port	Two autoregressive steps, two accumulated batches: maximum output, loss, gradient, and AdamW-update error all zero.
DISCO CPU contraction	Matches the library operator and first gradients; float64 reference error about 10^{-15}; finite-difference input gradient passes.
Spherical geometry	DISCO operator longitude shifts, scalar pole rows, and total quadrature weight 4π; full-model shifts aligned with coarsening.
Architecture	All nine spatial convolutions in the two-block compact model replaced; only 1×1 Conv2d layers remain in DISCO mode; no learned spatial bias factors.
Static-feature reuse	Losses and parameter gradients match recomputation for truncation lengths 1, 2, and 3.
Checkpoint continuation	Bitwise agreement for both backends; actual fit-loop interruption/resume crosses an epoch boundary.
Data and loss	Complete-window split isolation, one/two-frame order, normalization and cache immutability; all 97 named loss weights.
Native execution	Two CPU ranks with accumulation, validation and checkpointing; compiled forecast CLI agrees with eager physical fields, including horizon changes, incremental Zarr and lead-time coordinates.
Compilation	Full weather-shaped outputs and parameter gradients agree within the recorded tolerances before enabling compilation.

Table 1: Runnable checks are in `tests/`; compilation checks are in `experiments/tune_compile.py`. These are scoped checks, not evidence of untested GPU behavior.

Two upstream correctness changes are intentional and separate from the exact port fixture. First, the original loader could include targets beyond a training split's end date. Complete input/target windows are now contained in each split. Second, common-first channel reordering interleaves surface variables before some atmospheric outputs, so pressure weighting by position was incorrect. Weights now use feature names and pressure suffixes. For example, a surface wind received weight 0.2 instead of 1, while a 925-hPa vertical-velocity output received 0.1 instead of 0.0925. The upstream loss formula otherwise remains unchanged. A one-input shape bug and forecast output-frequency lead times are also corrected; existing forecast stores are protected from overwrite.

2.2 DISCO and CPU implementation

The adapter uses NVIDIA torch-harmonics at `6e7648d4435109c8a4ae747a9cd279b6ae3a30ba` [2, 3]. For an odd spatial width k, its piecewise-linear basis shape is $(k, 2k+2)$, yielding k^2 learned basis coefficients. The angular support radius is $k\Delta\phi/2$. Grids include both latitude poles, use an even periodic longitude count, and preserve these conditions through coarsening. The adapter reverses the south-to-north data convention for the library and averages scalar pole values. These checks establish the implemented longitude symmetry; they do not establish full rotation equivariance of the weather model. Scalar pole averaging is an operator property: inherited local average pooling can reintroduce longitude variation at the full model's pole outputs. This limitation is recorded in the full-model diagnostic; the pooling rule is retained for the upstream comparison.

Profiling identified basis-expanded depthwise activations and copies as a CPU bottleneck. A small C++ operator directly contracts the same quadrature stencil, without changing the learned basis or geometric weights. Its backward assigns each channel to one worker, avoiding atomic gradient accumulation. The path is used only for CPU float32/float64 depthwise layers. Other cases use torch-harmonics. Build flags enable OpenMP, vectorization, and the current CPU architecture. Higher-order derivatives are outside the implemented extension interface.

The retained kernel microbenchmarks show roughly 3.3× and 2× training speedups over the library implementation for the tested large and small layer shapes. These isolated operator gains are not end-to-end model speedups.

Library fusion and channels-last layout were evaluated and did not consistently improve both shapes, so they remain disabled. Compilation was retained for the paired weather runs after numerical checks. Its initial cost was 164–265 seconds in the pilot and is reported separately from warmed latency.

A subsequent CPU optimization folds overlapping basis contributions for each input point before applying them to the longitude vector:

$$\sum_{k,j} w_{ck}\psi_{khj}x_{cj} = \sum_{j}\left(\sum_{k} w_{ck}\psi_{khj}\right)x_{cj}.$$

This removes repeated activation reads and reuses the same dot product for weight gradients. The prototype matched values and first gradients before adoption; full-model, resume, forecast, and compilation checks were then repeated. The training processes retained the first direct kernel already loaded in memory. Final performance measurements and checkpoint re-evaluation use the folded kernel; source hashes and this revision distinction are preserved in the result files. Aggregate checkpoint scores are checked against the original logged validation losses to bound the numerical effect on the reported weather results.

Kernel shape	First direct kernel (ms)	Folded kernel (ms)	Speedup
17×32, C=3	31.190	31.287	1.00
61×120, C=32	86.191	82.086	1.05
121×240, C=64	334.881	183.407	1.83

Table 2: Final interleaved C++ forward-plus-backward microbenchmarks, four CPU threads. The 17×32 check uses float64 and batch 2; weather shapes use float32 and batch 1. Kernel speedups are not whole-model speedups.

3 Tracer reconstruction

The grid contains 721 latitudes and 1,440 longitudes. The initial tracer is an isotropic geodesic Gaussian with amplitude 1 and standard deviation 5°, centered at $(0°, 0°)$. Its analytic target follows a 68° great-circle displacement with initial northeast bearing 45°. These initialization settings are explicit fallback assumptions where original experiment details were unavailable. The objective uses exact latitude-cell areas, including half-cells at the poles:

$$\mathcal{L} = \frac{\sum_{ij} A_i(\hat{q}_{ij} - q_{ij})^2}{\sum_{ij} A_i}.$$

The reported relative MSE divides this by the zero-predictor MSE, preventing a small absolute loss on a mostly empty globe from being mistaken for successful transport. Signed mass error and prediction extrema are also in the raw results. The optimal spatially constant prediction has amplitude 0.00380 and relative MSE 0.99241. Reaching this baseline, even with accurate total mass, does not demonstrate localization or transport of the Gaussian.

The compact full PARADIS has 8 latent, 4 transported, and 8 static channels, two processor blocks, and spatial coarsening by 2. It receives the tracer and sine of latitude; the U-Net receives the tracer alone. A depth-three U-Net with top width 4 is reconstructed from the paper's Figure 19: an entry convolution followed by a normalized two-convolution residual block at each level. The chosen 3×3 kernels, upstream ChannelNorm, unit residual scale, average pooling, and bilinear upsampling remain implementation assumptions. This version has 46,481 parameters, versus the paper's stated 45,909; the original toy-experiment code was not recovered. Earlier plain double-convolution U-Net runs (30,601 parameters) are retained as auxiliary diagnostics and excluded from the main figure. Both models use seed 42, AdamW with learning rate 0.002, zero weight decay, and gradient clipping at 1. These runs fit one initial Gaussian; they do not test generalization to unseen tracer fields or held-out displacements.

The main protocol trains directly against the fixed 68° target for up to 500 updates. Separately labelled diagnostics (i) ramp the target over 300 updates then hold it fixed, and (ii) first train identity transport for 300 updates, ramp over 600 updates, then hold the target fixed. These are optimization diagnostics, not substitutions for the main fixed-target result. Time caps and completed update counts are preserved in each JSON record.

Under the fixed-target protocol, the residual U-Net converges to an almost constant field. PARADIS produces a broad zonal band near the target latitude, with only weak longitude variation. Its error closely matches the optimal longitude-averaged target (relative MSE 0.93466). Neither reconstructs the localized Gaussian: the modest reduction in PARADIS's relative MSE does not demonstrate successful transport. The separate identity-first diagnostic tests whether a gradual target change can avoid this optimization failure.

Model/protocol	Parameters	Updates	Angle	MSE	Relative MSE	Minutes
PARADIS + warmup	3,562	1000	68.0	4.41e-06	0.002	147.7
PARADIS + ramp	3,562	204	46.2	0.00207	1.091	90.3
PARADIS	3,562	500	68.0	0.00178	0.935	77.8
U-Net(P) + warmup	30,601	1000	68.0	0.00189	0.992	58.7
U-Net(P) + ramp	30,601	400	68.0	0.00189	0.992	20.9
U-Net(R) + warmup	46,481	1000	68.0	0.00189	0.992	145.9
U-Net(R) + ramp	46,481	400	68.0	0.00189	0.992	52.6
U-Net(R)	46,481	500	68.0	0.00189	0.992	65.3
U-Net(P)	30,601	500	68.0	0.00189	0.992	26.4

Table 3: Every attempted tracer protocol is retained. U-Net(R) is the residual reconstruction and U-Net(P) is the earlier plain diagnostic. "Ramp" denotes angular continuation; "warmup" adds identity training. Elapsed optimization time includes final evaluation, but excludes model/stencil construction. Angle is the last training target in degrees; a time-capped run may not reach 68°.

With identity warmup and gradual displacement, PARADIS does localize the Gaussian at the final 68° target: relative MSE is 0.00232 and peak amplitude is 0.989. The signed mass error is nevertheless +8.61%, and small negative values occur. This is a successful field fit under the modified optimization protocol, not exact conservative transport. The low MSE alone would hide the contribution of a weak background over the rest of the globe.

4 Partial weather comparison

Real ERA5 data comes from the WeatherBench 2 121×240 equiangular-with-poles store [4]. Training uses all of 2016 (1,464 six-hourly times), and validation uses January 2017 (124 times). Two input frames and one forecast step yield 1,462 training and 122 validation windows. The full configured pressure-level/variable selection gives 97 output channels. Cartesian wind conversion follows upstream; physical constants and fractional land–sea mask are retained. Lossless storage and training-only normalization statistics avoid an additional quantization change. The source URI, coordinate checks, and preparation provenance are in `results/data-manifest.json`.

Both optimized models have 111,638 parameters, two processor blocks, 64 latent, 32 transported, and 32 static channels. Training is paired by seed, sample order, update count, AdamW hyperparameters, and learning-rate schedule. The initial learning rate is 5×10^{-4}, Adam betas are $(0.9, 0.95)$, and weight decay is 0.01. Twenty warmup updates precede the steady phase; the last 20% decay linearly. Every 200 updates, all 122 validation windows are evaluated using the configured latitude-, pressure-, and variable-weighted reversed-Huber loss. The common count of 2,000 updates was fixed before comparative training.

Backend	Updates	Final validation	Best validation	Fit minutes
conv2d	2,000	0.11408	0.11408	61.0
disco	2,000	0.11402	0.11402	90.4

Table 4: Paired short weather runs. Fit time includes compilation, training, validation, and checkpoint writes, and excludes dataset preload/model construction. Other tracer jobs ran concurrently; these fit times are descriptive.

The final DISCO/Conv2d validation-loss ratio is 0.9995. The predeclared *descriptive* comparability threshold is 1.10, which this result meets. This is one short, single-seed comparison, approximately 1.37 passes over the training windows; it is not a statistical equivalence test or evidence of full-scale forecasting skill. The short run, changed architecture, resolution, coordinate inputs, and corrected loss weighting prevent direct comparison with the paper's published scores.

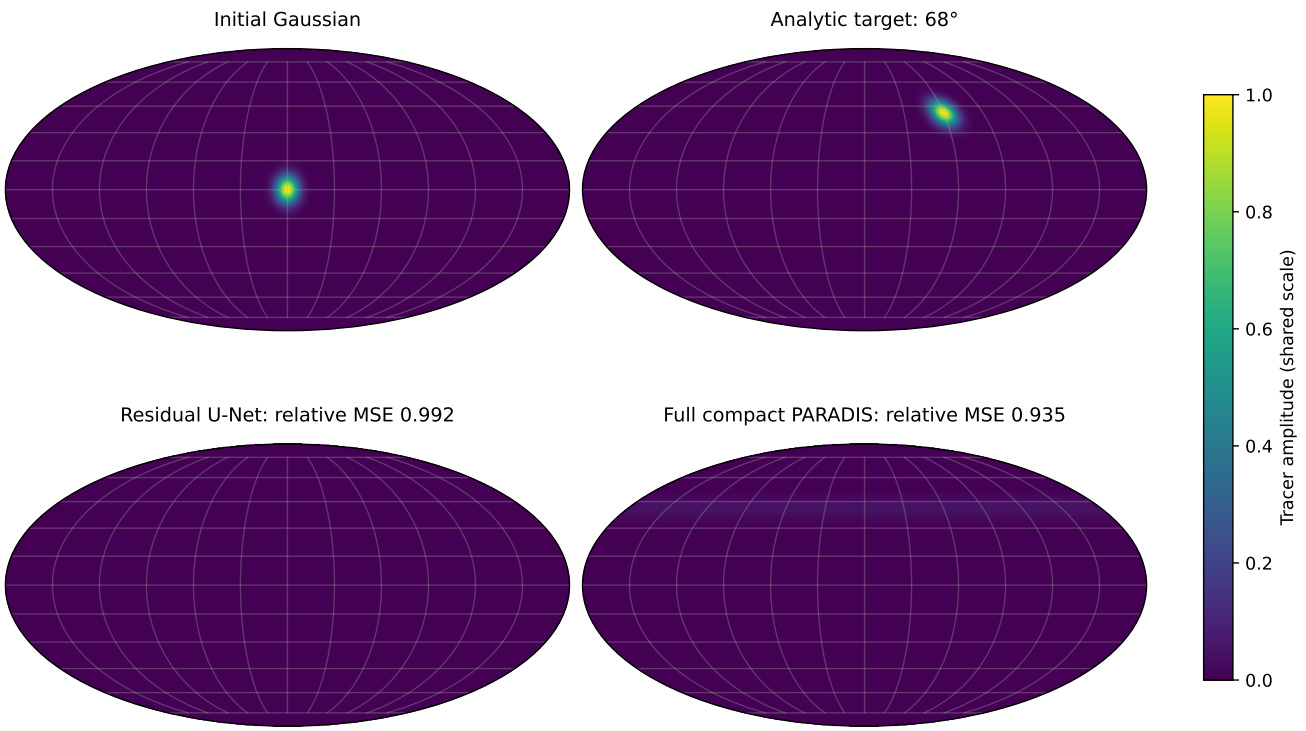


Figure 1: Figure 3-style reconstruction under the fixed-target protocol. All maps use the same amplitude scale and the full grid; the lower-right panel is the requested compact full PARADIS with DISCO. A relative MSE near 1 indicates little improvement over predicting zero. Negative/above-one predictions are outside the shared display range and are quantified in the raw metrics.

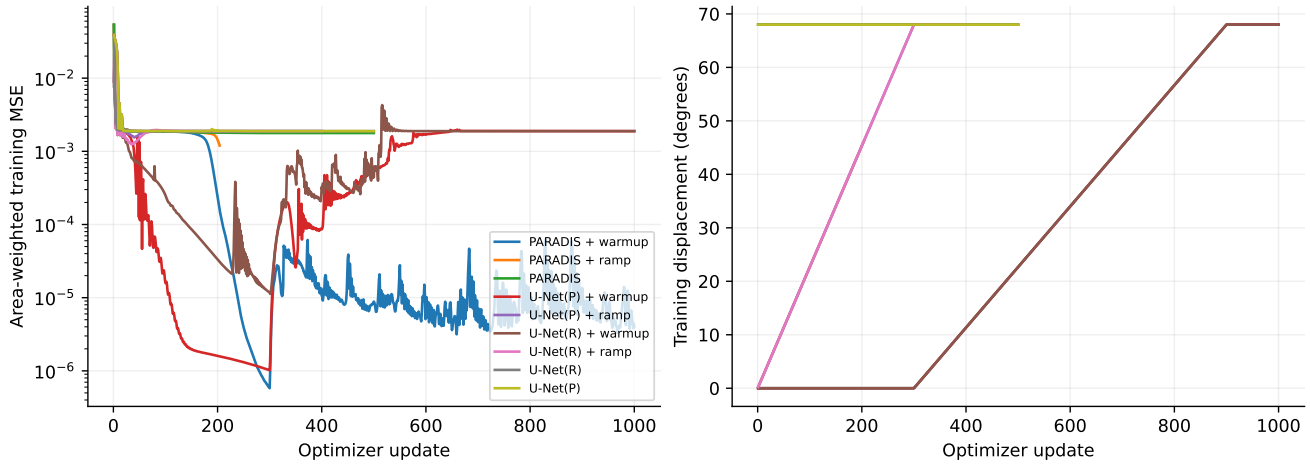


Figure 2: Tracer optimization histories and the actual target angle at each update. Losses during continuation refer to the moving training target; final table errors always use the 68° target.

Predictor	z_{500} RMSE ($m^2\,s^{-2}$)	T_{2m} RMSE (K)
persistence	236.52	2.536
conv2d	167.87	3.174
disco	170.14	3.129

Table 5: Six-hour RMSE of final checkpoints, pooled across all 122 January validation windows with cosine-latitude weighting. Persistence carries the last input field forward. These physical-unit scores contextualize the aggregate loss; they are not the paper's long-lead forecast evaluation.

Both short-trained models improve on persistence for geopotential and are worse than persistence for 2 m temperature. Similar aggregate validation loss therefore does not establish uniform forecasting skill across variables.

5 Performance measurements and limits

The host is a dual-socket Intel Xeon Gold 5220R (48 physical cores, 96 logical CPUs), with approximately 754 GiB RAM. The CPU environment uses Python 3.11.17 and PyTorch 2.14.1+cpu. The final latency experiment uses four pinned CPU threads, with our other training jobs finished. External host load and operating-system noise remain uncontrolled.

The benchmark compares the pinned upstream model, optimized Conv2d, and optimized DISCO using identical full-feature synthetic inputs, grid 121 × 240, batch size 1, two blocks, 64 latent/32 transported channels, and matched static widths 128/64. The upstream model retains its spatial bias maps. The input width is held fixed to isolate execution; this is not the complete original coordinate configuration. Training includes forward, loss, backward, and AdamW in the same native harness, not the Lightning framework. The optimized variants are compiled; upstream is eager. Two warmup calls per workload/variant and construction/compile time are reported outside the steady-state measurements.

Each measurement block randomizes variant order. Twenty blocks are retained. Bootstrap resampling of paired blocks (10,000 draws) estimates a one-sided 95% upper bound on the ratio of median latencies. The strict no-slowdown criterion is an upper bound at most 1 for *both* inference and training. It is a gate for this CPU workload, not a universal performance guarantee.

The inference upper bound is 0.897; the training upper bound is 0.815. The joint no-slowdown criterion passes for this workload. The JSON `no_slowdown` fields retain both individual outcomes. Relative to optimized Conv2d, DISCO takes 1.84× the inference time and 0.90× the training time. The sample distributions and bootstrap intervals are broad, so the median speedups should be read as workload- and host-specific measurements. A

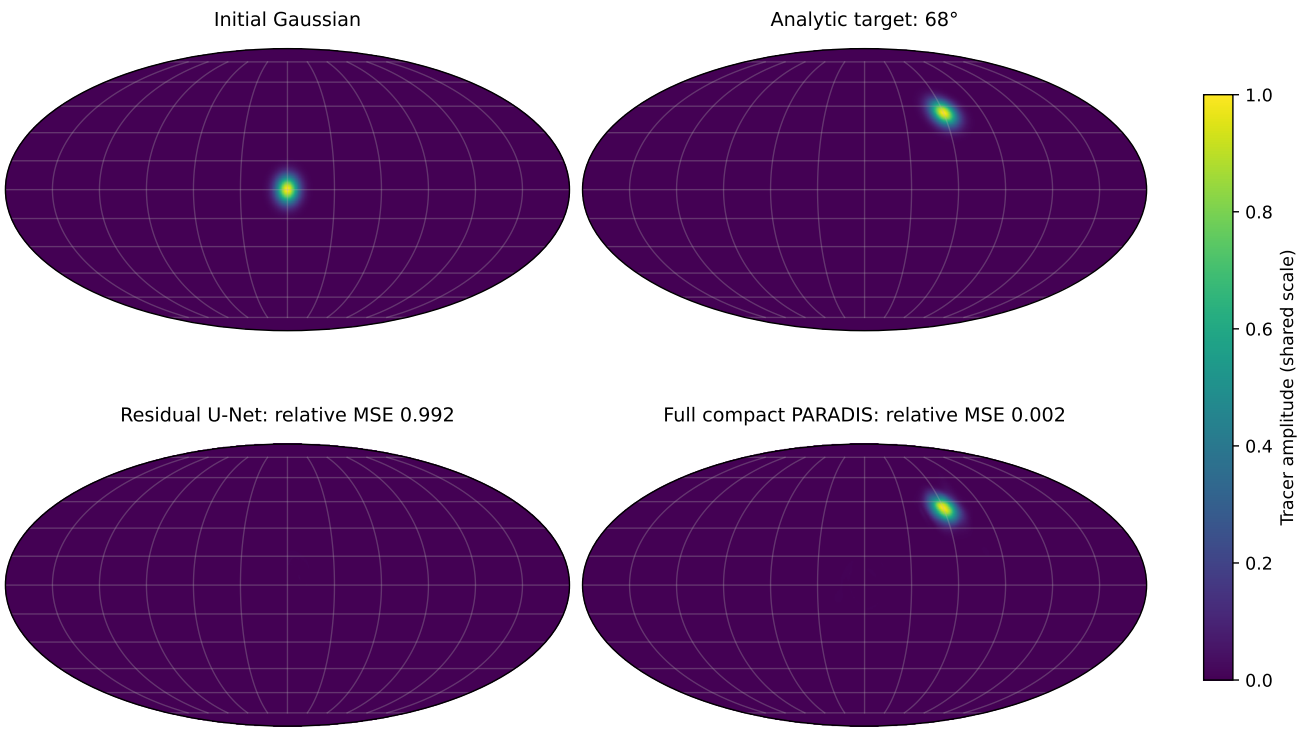


Figure 3: Separate identity-warmup and angular-continuation diagnostic, using the same display scale as the fixed-target figure. Training starts at zero displacement, then increases it toward 68°. This explicitly modified optimization protocol tests whether the full architecture can track transport after learning the field representation; it is not an exact repetition of the paper's fixed-angle procedure.

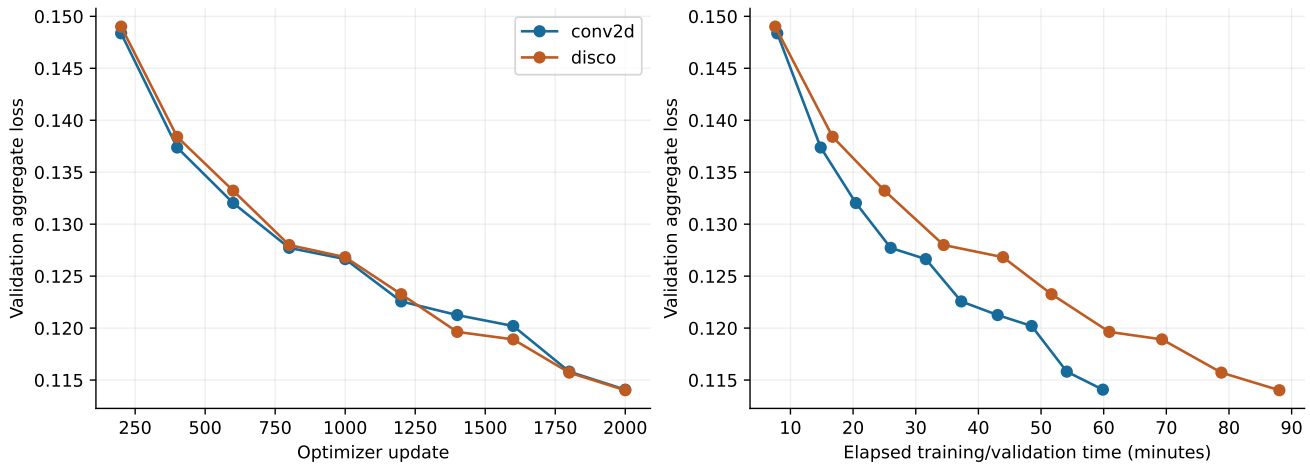


Figure 4: Validation loss versus equal update count and elapsed time. Logged time is sampled immediately before each validation pass, so each point excludes that pass's duration; the table uses total fit wall time.

Workload	Variant	Median seconds	DISCO/upstream upper 95%
inference	upstream	1.9688	—
inference	conv2d	0.5495	—
inference	disco	1.0132	0.897
training	upstream	3.6989	—
training	conv2d	2.7503	—
training	disco	2.4721	0.815

Table 6: Warmed, interleaved final measurements. Bounds are shown only on the DISCO rows. Raw samples and both upstream/control comparisons are committed.

failed bound is retained as a failed target, even when the point estimate or an isolated kernel is faster. No CPU result establishes GPU parity.

Variant	Construction (s)	Inference warmup (s)	Training warmup (s)
upstream	0.1	11.9	24.6
conv2d	0.1	75.5	187.0
disco	21.7	55.0	155.9

Table 7: Observed construction and two warmup calls per workload. Optimized variants include compilation costs. Compiler caches contain earlier trials; these are observed setup costs, not a clean-cache compiler benchmark.

Fresh-process memory measurements in `results/memory-*.json` report resident peaks including Python, libraries, model construction, and compilation. They are not isolated tensor-allocation peaks; compiler subprocess memory is not included.

The separate data-loading benchmark compares preloaded January samples at matching initialization times and checks numerical agreement of dynamic fields and forcings. Constant coordinate channels differ intentionally. Setup timings include reads with an uncontrolled filesystem cache. Median sample time is 391.4 ms upstream and 260.1 ms optimized, a measured speedup of 1.50×; dynamic/forcing values agree exactly. Pilot profiles and tuning trials are retained for audit; the earliest pilot used only five output features and must not be compared directly to the final 97-output-channel latency measurements.

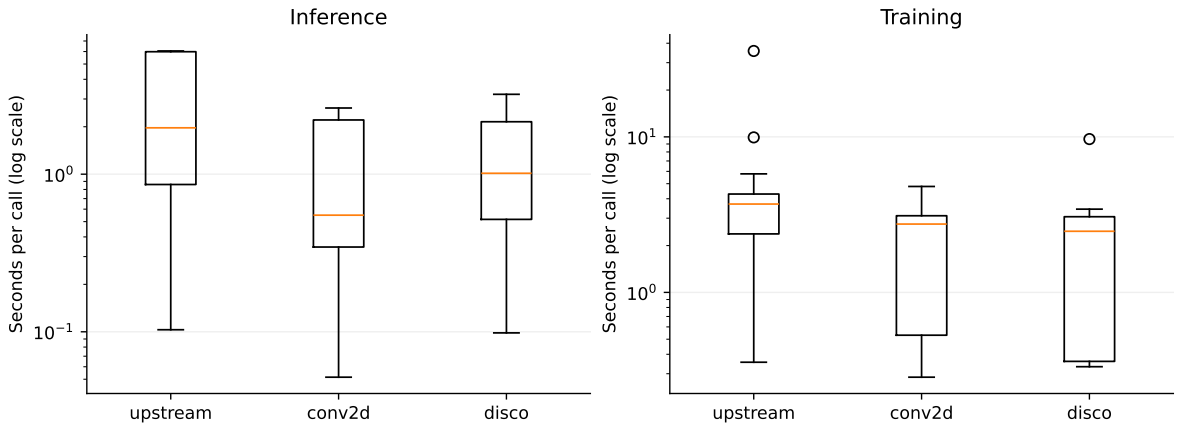


Figure 5: Final latency sample distributions on logarithmic axes, retaining every sample and outlier. Compilation and setup are excluded from these boxes and retained separately in the raw result file.

Variant	Import peak (GiB)	Full process peak (GiB)
upstream	0.36	1.16
conv2d	0.36	1.02
disco	0.36	1.04

Table 8: Separate processes, three training updates each, four CPU threads; optimized variants compiled, upstream eager.

6 Reproduction and remaining limitations

The README provides commands for environment construction, data preparation, training, forecast writing, each regression check, benchmarks, and tracer runs. Rebuild figures and PDF with:

```
python report/make_figures.py
cd report
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

The existing locked environment was exercised, but a second clean installation was not used as a verification step. Dataset stores and model checkpoints remain local and are excluded from Git; configurations, histories, numerical summaries, figures, source, and the PDF are committed. The full-resolution tracer field files are available in the local data directory and generated by the experiment driver.

This work verifies a native CPU implementation and a bounded modified experiment. The full upstream training scale, original toy-model code, multiple training seeds, long-range weather skill, and all CUDA paths remain unverified. These limitations must accompany any reuse of the reproduction's numerical comparisons.

References

[1] Pereira et al. *Learning to Advect: A Neural Semi-Lagrangian Architecture for Weather Forecasting.* arXiv:2601.21151v1. https://arxiv.org/html/2601.21151v1.

[2] NVIDIA. *torch-harmonics*, pinned source revision listed above. https://github.com/NVIDIA/torch-harmonics.

[3] Ocampo, Price, and McEwen. *Scalable and Equivariant Spherical CNNs by Discrete-Continuous (DISCO) Convolutions.* arXiv:2209.13603. https://arxiv.org/abs/2209.13603.

[4] Google Research. *WeatherBench 2 datasets.* https://weatherbench2.readthedocs.io/en/latest/data-guide.html.